

SpaceRacer Technical Materials

Hardware and Software Stack for Autonomous 1:10 Racing

1. Team Information

Team name	SpaceRacer
Organization	Shenzhen University
Team leader	Haorong YUAN (2510095081@mails.szu.edu.cn)
Team member	Shengyao LAN (dhirenoncaloricfp@gmail.com)
Registration contact	2510095081@mails.szu.edu.cn

2. System Overview

SpaceRacer is an autonomous racing vehicle built for the Roboracer competition objective: avoiding crashes while minimizing lap time. The vehicle uses a single-LiDAR perception setup, particle-filter localization, global path tracking, and an MPC-based local avoidance layer. The implementation is designed to keep the stack simple, robust, and suitable for real-car deployment.

Layer	Main Components	Purpose
Sensing	Livox Mid-360 LiDAR; VESC odometry	Obstacle perception and vehicle motion feedback
Localization	Particle Filter on a pre-built map	Estimate vehicle pose for path tracking
Planning	Centerline waypoints and local avoidance targets	Provide reference path and safe local target
Control	Pure Pursuit and MPC tracking	Generate steering and speed commands

3. Vehicle Hardware Specification

Registered Vehicle Configuration	
Chassis platform	ARRMA Mojave EXB(Width:340mm;Length :550mm)
Battery	3S LiPo x 1
LiDAR sensor	Livox Mid-360
Motor	EZRUN-4268SD-2500KV x 1
ESC	VESC MK VI
Powertrain	Single motor unit with electronic speed control

4. Software Stack

The software stack is organized as a modular ROS-based autonomy pipeline. The core runtime data flow is:

Livox Mid-360 / VESC feedback -> perception and localization -> global reference path -> MPC_avoidance local planner -> MPC or Pure Pursuit control -> VESC drive command.

Module	Input / Output	Description
Map and Waypoints	Input: pre-built map and centerline waypoints; Output: reference path	The global route is represented by waypoints and used as the nominal racing reference.
Localization	Input: LiDAR and map; Output: vehicle pose	Particle Filter estimates the car pose in the map frame for closed-loop racing.
Path Tracking	Input: pose and global reference; Output: steering and speed command	Pure Pursuit is retained as a baseline controller. MPC is used for smoother and constraint-aware tracking.
Obstacle Handling	Input: LiDAR scan and reference target; Output: safe local target or slow/stop decision	MPC_avoidance checks whether the direct path to the lookahead target is blocked and searches for a collision-free local target.

5. Localization Algorithm

The implemented localization algorithm is Particle Filter. It estimates the vehicle pose by matching LiDAR observations with a pre-built occupancy grid map, pre-built occupancy grid map is generated using slam_toolbox in ROS 2. . The output pose is used by the controller and planner as the main state feedback. This method is preferred because it is lightweight, robust on small racing tracks, and compatible with real-time F1TENTH-style racing deployment.

6. Control Algorithm

The control stack contains two controllers:

- Pure Pursuit: used as a baseline global waypoint/path tracking controller.
- Model Predictive Control (MPC): used as the main tracking controller for the selected global or local reference trajectory.

In normal driving, the vehicle follows the global reference path. When obstacles are detected, the controller receives a safe local target or short local trajectory from MPC_avoidance and tracks it with steering and speed constraints.

7. Safety and Constraints

- Only one LiDAR unit is used: Livox Mid-360.
- The battery configuration is 3S LiPo x 1, below the maximum 4S LiPo limit.
- The vehicle uses a single motor and one ESC.
- Speed reduction and emergency stop are triggered when no safe local target is available.
- Steering and velocity commands are limited before being sent to the VESC.

8. Detection and Avoidance: MPC_avoidance

The detection and avoidance module is named MPC_avoidance.

Step	Method	Output
1. LiDAR preprocessing	Convert the Livox Mid-360 perception result into a local obstacle representation in the vehicle frame.	Local obstacle points or grid cells
2. Local occupancy check	Build a short-range local occupancy grid and inflate occupied cells with a safety margin.	Collision-risk map
3. Path collision test	Check whether the line or corridor from the vehicle to the global lookahead target intersects occupied cells.	Free / blocked decision
4. Candidate target search	If blocked, laterally shift the target to the left and right, then evaluate collision-free candidate targets.	Best safe local target
5. MPC tracking	Generate a short local reference from the selected target and track it using MPC under steering and speed limits.	Drive command
6. Fallback	If no collision-free candidate is available, reduce speed or stop the vehicle.	Slowdown / emergency stop